Lezione 11 — Strategic DDD avanzato

Blocco 3 · AI Software Engineering · Dispensa di studio
Subdomain, Context Mapping e confini di integrazione · tempo previsto: 40–50 minuti tra lettura e laboratorio · materiale framework-independent

1. Obiettivi e prerequisiti

La lezione prosegue la pipeline **Domain → Requirements → Spec → Test → Agent → Verify**. Presuppone conoscenza di DDD, bounded context, invarianti, contract testing, context engineering e valutazione degli agenti. Il focus è sulle scelte strategiche che precedono il design di classi e servizi.

Alla fine saprai:

- distinguere Problem Space (subdomain) e Solution Space (bounded context);
- classificare Core, Supporting e Generic Subdomain e motivare dove investire;
- individuare confini semantici, relazioni e ownership;
- costruire una Context Map e scegliere i pattern di relazione in base ai tradeoff;
- produrre una specifica agent-ready con criteri di verifica indipendenti.

2. La domanda architetturale e l'esempio guida

Un sistema può avere controller, service e repository ben organizzati e tuttavia non sapere dove terminino le responsabilità di Operations e dove inizino quelle di Billing. Le directory tecniche non esprimono da sole ownership e linguaggio: il punto di partenza sono le distinzioni del business, non quelle imposte dal framework. Prima di far lavorare agenti AI bisogna quindi sapere *come dividere* il sistema.

Strategic e Tactical DDD non sono alternative. Il primo ragiona su aree del problema, modelli, relazioni e team; il secondo introduce entity, value object, aggregate, domain service ed eventi dentro uno specifico bounded context. Prima definiamo chi possiede quale modello, poi come realizzarlo. La Lezione 12 approfondirà il livello tattico.

> **Esempio guida: gestionale di manutenzione.** Una piattaforma riceve segnalazioni di manutenzione (Reporting), assegna e fa eseguire i lavori (Operations) e prepara i dati di fatturazione (Billing). Un tecnico può dichiarare un lavoro concluso: **non sappiamo** se questa dichiarazione basti per fatturare (UNKNOWN).

3. Perché non esiste un solo modello universale

La parola *intervento* non ha necessariamente un significato unico. Un modello universale rischia campi facoltativi, lifecycle incompatibili e logiche contraddittorie. Ogni contesto conserva il proprio linguaggio e scambia solo concetti concordati.

Contesto	Modello	Significato
Reporting	`MaintenanceRequest`	una richiesta da valutare
Operations	`WorkOrder`	lavoro assegnato ed eseguito
Billing	`BillableService`	prestazione economica

Lo stesso fenomeno si vede altrove: un *ordine* è per Sales un accordo commerciale, per Billing la fonte di elementi fatturabili, per Shipping la base della consegna. Gli stati dell'ordine possono divergere senza che nessuno dei tre contesti sia sbagliato. Evita quindi un unico oggetto universale `Intervention`.

L'obiettivo del Bounded Context è la **coerenza locale**: un linguaggio e un insieme di regole consistenti nel loro perimetro. Non significa pretendere uniformità aziendale globale.

4. Domain, Subdomain, Bounded Context

- **Domain:** area di conoscenza, processi e problemi dell'organizzazione.
- **Subdomain:** suddivisione di quell'area in capacità o responsabilità di business. Appartiene al Problem Space, ricavato dall'osservazione del business.
- **Bounded Context:** confine nel quale un modello e il suo linguaggio hanno significato coerente. Appartiene al Solution Space, cioè alla modellazione della soluzione.

La mappatura può essere uno-a-uno, uno-a-molti e talvolta molti-a-uno: un contesto può attraversare più subdomain, o un subdomain avere più modelli. I confini emergono da responsabilità, linguaggio e regole, non dalla struttura delle tabelle.

> **I confini non sono deployment.** Un bounded context non è automaticamente un microservizio, un database o un repository; un monolite modulare può contenere molti contesti. Regola pratica: non derivare un servizio per ogni subdomain. Traccia prima una mappa dei modelli e verifica dove il linguaggio cambia.

5. Core, Supporting e Generic

- **Core:** capacità che dà differenziazione competitiva; merita investimento in esperti, modellazione e verifica.
- **Supporting:** sostiene le operazioni, spesso richiede personalizzazione ma non è la principale differenziazione.
- **Generic:** problema comune che spesso si può acquistare o riutilizzare.

Nel gestionale di manutenzione, se il vantaggio consiste nel coordinare interventi e SLA, la pianificazione avanzata potrebbe essere Core; reportistica e anagrafiche Supporting; autenticazione e consegna email Generic. In un marketplace di servizi, un matching proprietario basato su competenze e vincoli è candidato Core, le notifiche standard sono Generic e le prenotazioni possono essere Supporting. Sono classificazioni esemplificative: serve una decisione motivata dai business goal.

Criticità ≠ differenziazione. Un componente Generic può essere security-critical (un sistema di autenticazione), mentre un Core può avere poco codice ma regole preziose. La classificazione dipende dal vantaggio strategico, non dal numero di righe di codice, e va rivista se cambia il business.

6. Scoprire i confini e governare la proprietà

Quattro domande per individuare un confine: chi decide la regola? dove lo stesso termine cambia significato? che cosa richiede consistenza (quali invarianti vanno mantenute insieme)? che cosa evolve indipendentemente? Aiuta anche chiedersi quali gruppi hanno conoscenza e roadmap diverse.

Ownership. Gli agenti devono ricevere le responsabilità già chiarite. Distinguiamo:

- **Business authority:** chi può approvare le policy;
- **Model ownership:** chi definisce linguaggio, stati e invarianti;
- **Contract ownership:** chi pubblica e governa le interfacce;
- **Data authority:** chi è fonte autorevole di una informazione;
- **Change governance:** chi approva i breaking change;
- **Operational ownership:** chi garantisce esercizio e sicurezza.

Il proprietario tecnico del codice non è necessariamente proprietario della regola di business. Nell'esempio guida: Reporting possiede `RequestStatus`, Operations `WorkOrderStatus`, Billing le regole di `BillableService`. Il proprietario del contratto pubblicato non autorizza interpretazioni arbitrarie nel consumer.

Per ogni contesto conviene documentare: scopo, responsabilità, linguaggio, input, output, assunzioni, invarianti, stakeholder e dipendenze. Il *Bounded Context Canvas* di DDD-Crew offre una struttura collaborativa per farlo.

7. Context Mapping e direzionalità

Una Context Map rappresenta la relazione tra contesti con un pattern esplicito. Non è solo una mappa di chiamate REST, di eventi Kafka o di dipendenze tra package: mostra quanto modelli e team dipendono l'uno dall'altro e chi influenza le decisioni semantiche. Registra anche owner, versione dei contratti e incognite. Preferisci mappe piccole che rispondano a domande specifiche, e mantienile vive: non è un diagramma una tantum.

Upstream è il contesto che influenza il downstream; **Downstream** è quello il cui modello o comportamento è influenzato. L'orientamento upstream/downstream non coincide necessariamente con la direzione di una chiamata HTTP.

8. Pattern di relazione

Partnership

I due team condividono un destino di delivery e pianificano insieme l'evoluzione delle interfacce. Esempio: chi pianifica le attività e chi assegna i tecnici possono condividere una release di business. Pro: collaborazione forte, compromessi rapidi. Contro: rilasci coordinati e maggiore dipendenza organizzativa. Dichiararla senza capacità organizzativa crea colli di bottiglia: non va usata solo perché due sistemi dialogano.

Customer–Supplier

L'upstream fornisce un servizio e ascolta le necessità del consumer, che può negoziare capacità e requisiti. Esempio: Reporting pubblica l'evento `RequestAccepted` che Operations deve consumare; Operations negozia le informazioni necessarie ma non può alterare gli stati interni del producer.

Relazione asimmetrica, a differenza della Partnership. Contro: conflitti di priorità. Ownership e governance dei breaking change devono essere esplicite.

Conformist

Il downstream accetta il modello upstream, limitando o eliminando la traduzione. Esempio: un sottosistema di reportistica non differenziante recepisce gli stati di un vendor. Pro: soluzione economica e semplice. Contro: dipendenza semantica; se l'upstream cambia, il downstream potrebbe dover seguire. Non è una debolezza automatica ma un *compromesso consapevole*, accettabile quando il modello esterno va bene o il modello locale non richiede autonomia.

Anti-Corruption Layer (ACL)

Il downstream protegge il proprio modello con una traduzione esplicita al confine. Traduce il **significato**, non solo il formato; è adatto a legacy, vendor e integrazioni non controllate. Contro: più codice e test di mapping, gestione di ambiguità, stati mancanti ed errori. Non è un semplice adapter che rinomina proprietà, e non deve inventare equivalenze tra stati di business.

```
ExternalJob(status="DONE")
   -> ACL -> externalCompletionReported = true
   != billable = true        // policy UNKNOWN
```

Se il sistema esterno invia `DONE` non possiamo assumere che il lavoro sia verificato amministrativamente e fatturabile: l'ACL lo traduce in `externalCompletionReported`, e la decisione di fatturabilità spetta a Billing e alle sue policy confermate. Analogamente `PAID` di un provider di pagamento può indicare un esito tecnico, mentre `Settled` può richiedere fondi regolati: stati ambigui ⇒ UNKNOWN, mai promozione arbitraria.

Shared Kernel, Open Host Service, Published Language, Separate Ways

- **Shared Kernel:** sottoinsieme piccolo e deliberatamente condiviso di modello (spesso anche di implementazione); ogni variazione richiede accordi. Un kernel di centinaia di entità condivise è segnale di accoppiamento eccessivo.
- **Open Host Service:** servizio che espone un protocollo stabile a più consumer.
- **Published Language:** modello di scambio pubblico e documentato. Complementare all'OHS, non equivalente: usare OpenAPI non basta per avere un Published Language semanticamente solido.
- **Separate Ways:** evitare o interrompere un'integrazione quando i costi di dipendenza non sono giustificati; una duplicazione circoscritta può costare meno di una sincronizzazione permanente.

Big Ball of Mud descrive invece confini scarsamente affidabili, da contenere.

Come scegliere

Non esiste un pattern migliore in assoluto. Modelli naturalmente compatibili → valuta Conformist; modello locale da proteggere → ACL; forte reciprocità → Partnership; influenza reale del consumer sull'upstream → Customer–Supplier; interazione costosa e inutile → Separate Ways. Una scelta proporzionata al costo di dipendenza e al valore da proteggere.

9. Approfondimento SaaS: dal provider alla subscription

Una piattaforma riceve da un provider il payload `event=payment.succeeded, payment_id=pay_123, amount=2900, currency=EUR`. Prima di attivare un abbonamento occorre valutare autenticità, identità

del contratto, importo previsto, valuta, idempotenza, storni e policy di attivazione. `payment.succeeded` non implica automaticamente che la subscription vada attivata.

Proposta architetturale: un adapter gestisce protocollo e autenticità della comunicazione; l'ACL produce una `PaymentEvidence` nel linguaggio locale; l'application service valuta le regole del contesto Subscription. Il provider non stabilisce le politiche del servizio: il webhook non è una business policy.

10. Verifica attraverso i confini

- **Contract test:** formato e comportamento dei messaggi ricevuti;
- **Property test:** invarianti, duplicati e idempotenza, quando richiesti;
- **Acceptance test:** regole di business approvate dagli stakeholder;
- **Eval agentica:** l'agente non deve inventare policy né decidere senza evidenza.

11. Agent-assisted Strategic DDD

Gli agenti possono estrarre nomi, processi e dipendenze da codice e documenti; proporre Subdomain Map, Context Map, Bounded Context Canvas e ADR; rilevare conflitti terminologici; trovare i punti in cui i contratti si propagano. Ogni conclusione deve conservare la **provenienza delle evidenze**. Le mappe prodotte dall'agente restano in stato `CANDIDATE` fino alla revisione umana.

Workflow: grill-with-docs / grill-me → domain-modeling → glossario / ADR → to-spec → to-tickets → implement/TDD → code-review. Sono nomi di skill e convenzioni pratiche, non sostituti del DDD; funzionalità e naming possono cambiare.

I risultati devono separare **KNOWN**, **ASSUMED**, **UNKNOWN** e **DECIDED**. Un agente non può promuovere un'ipotesi a decisione né scegliere automaticamente una topologia di microservizi. Catena: revisione umana → ADR → Spec → Test.

Task agent-ready

Un task per un coding agent contiene: contesto, decisioni approvate, input e vincoli; le open question vengono preservate come UNKNOWN; l'output atteso è adapter, mapping, contract test e verifica. Esempio: *realizza il consumer dell'evento `WorkCompleted v1` in Billing senza importare la persistenza di Operations, validando schema e idempotenza solo dopo aver deciso la relativa policy.* L'agente può proporre opzioni, ma non decidere il significato di dati non chiariti.

```
DECIDED:   Operations possiede WorkOrderStatus
DECIDED:   Billing possiede la billability policy
UNKNOWN:   il completamento tecnico è sufficiente per fatturare?
FORBIDDEN: import diretti dei model di Operations
```

12. Anti-pattern e failure mode

Il rischio non è solo tecnico: test e build possono risultare verdi pur avendo incorporato regole semanticamente errate. Riesamina periodicamente mappa, contratti, codice e ownership.

- Un contesto per ogni tabella; ogni funzione dichiarata Core; Core uguale a «massimo numero di classi».
- Un microservizio per contesto per default; microservizi prima del domain modeling.
- Conformist involontario nel Core; Shared Kernel gigante.

- ACL indiscriminati, o ACL che inventa business policy.
- Mappe obsolete, boundary senza owner, regole di business suggerite dall'AI e mai validate.

13. Laboratorio progressivo

Scenario. Una piattaforma riceve segnalazioni di manutenzione, assegna i lavori e prepara i dati di fatturazione. Un tecnico può dichiarare lavoro concluso; *non sappiamo* se questa dichiarazione sia sufficiente per fatturare: UNKNOWN.

A. Subdomain e confini

Elenca Core, Supporting e Generic motivando la classificazione alla luce del vantaggio competitivo ipotizzato. Definisci tre bounded context (Reporting, Operations, Billing) e, per ciascuno, glossario locale, responsabilità, fonte autorevole e policy esplicitamente sconosciute.

B. Context Map

Disegna due integrazioni: Reporting → Operations e Operations → Billing. Per ciascuna documenta producer, consumer, dipendenza upstream/downstream, pattern candidato, contratto, traduzioni e owner. Distingui le ipotesi dalle decisioni approvate.

C. Artefatti

Crea, in stato PROPOSED: `docs/domain/SUBDOMAINS.md`, `docs/domain/GLOSSARY.md`, `docs/domain/CONTEXT-MAP.md`, `docs/decisions/ADR-001-integration-boundaries.md` e `docs/contracts/work-completed-v1.schema.json`.

D. Verifiche indipendenti

- **Test 1:** nessun import di classi di persistenza di Operations nel codice di Billing.
- **Test 2:** ogni evento in ingresso è conforme al contratto nella versione dichiarata.
- **Test 3:** nessuna trasformazione automatica di `WorkCompleted` in autorizzazione alla fatturazione senza policy esplicita.

E. Accettazione e rubrica

Il lavoro è completo se ogni decisione è tracciabile, gli UNKNOWN restano aperti, la mappa distingue responsabilità e relazioni, e almeno tre controlli possono essere svolti indipendentemente dall'agente implementatore. Criteri di valutazione: confini giustificati con linguaggio e responsabilità; strategia distinta da criticità operativa; dipendenza semantica distinta dalla direzione HTTP; evidenze separate da assunzioni; decisione motivata con alternative, conseguenze e rischi.

> **Variante: marketplace di servizi.** Un marketplace offre catalogo, disponibilità, prenotazioni, pagamenti, assegnazione professionisti, notifiche, recensioni. L'elemento competitivo è un algoritmo proprietario che usa disponibilità, competenze e geografia; pagamenti e notifiche dipendono da servizi esterni standard.
>
> - **Task A:** classifica i candidati subdomain Core, Supporting e Generic e spiega perché.
> - **Task B:** proponi almeno quattro Bounded Context con responsabilità e linguaggio; rappresenta pattern di relazione, upstream/downstream e almeno due UNKNOWN da chiarire.
> - **Task C:** scrivi un ADR per decidere se Booking accetta gli stati del provider di pagamento (Conformist) o li traduce con un ACL.

Gli scenari di manutenzione, SaaS e marketplace sono esercizi didattici, non decisioni su progetti reali; non si presume che il laboratorio sia stato eseguito, e nessun progetto reale è assunto come laboratorio senza autorizzazione.

14. Autovalutazione

- **Un subdomain coincide sempre con un bounded context?** No: descrivono piani concettuali differenti (Problem Space vs Solution Space).
- **Conformist è sempre sbagliato?** No: può minimizzare il costo di integrazione quando il modello upstream è accettabile.
- **Un ACL è solo mapping JSON?** No: è una protezione semantica.
- **Un evento di completamento garantisce la fatturabilità?** Non senza una business policy esplicita.
- **Una Context Map è un artefatto statico?** No: va mantenuta allineata a contratti e responsabilità.
- **Upstream significa «chi chiama»?** No: indica chi influenza il modello, non la direzione del traffico di rete.

15. Sintesi, connessioni e prossimi passi

Domain → Requirements → Spec → Test → Agent → Verify: lo Strategic DDD informa soprattutto la modellazione iniziale e la proprietà dei contratti; i test e le eval successivi verificano che tali confini non vengano violati. **Takeaway:** il dominio governa la forma dei confini; il confine semantico non coincide con la topologia del deploy; si cerca coerenza locale, non uniformità globale; gli agenti implementano relazioni decise, non inventano policy.

La **Lezione 12** passerà dal livello strategico ai consistency boundaries: Entity, Value Object, Aggregate Root, Domain Service e Domain Event.

16. Fonti

- Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, 2003.
- Vaughn Vernon, *Implementing Domain-Driven Design* (2013); *Domain-Driven Design Distilled* (2016).
- Martin Fowler, *Bounded Context* — martinfowler.com/bliki/BoundedContext.html
- Microsoft Learn, *Use domain analysis to model microservices* — learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis
- DDD-Crew, *Context Mapping* — github.com/ddd-crew/context-mapping; *Bounded Context Canvas* — github.com/ddd-crew/bounded-context-canvas
- Context Mapper — contextmapper.org/docs/language-reference/ e contextmapper.org/docs/anticorruption-layer/